MODULE *DataLakehouseGateway*

EXTENDS *Naturals*, *Sequences*, *FiniteSets*, *TLC*

CONSTANTS
Tenants, *Clients*, *Requests*, *Queries*, *Workers*, *Blobs*,
MaxSnapshot, *MaxAttempts*, *MaxCrashes*, *MaxTimeouts*,
RequestedTenantOf, untrusted tenant from caller payload
AuthenticatedTenantOf, tenant from verified auth (*e.g. JWT*)
QueryOf, *ClientOf*,
ProtocolOf, "FLIGHT" (sync, in-VM) or "REST" (async)
SecurityContextOf,
FlightPopulatesCache, BOOLEAN : should Flight results enter cache?
MutantSkipTenantCheck BOOLEAN : FALSE normally; TRUE = inject bug

ASSUME
$\land$ DOMAIN $RequestedTenantOf = Requests$
$\land$ DOMAIN $AuthenticatedTenantOf = Requests$
$\land$ DOMAIN $QueryOf = Requests$
$\land$ DOMAIN $ClientOf = Requests$
$\land$ DOMAIN $ProtocolOf = Requests$
$\land$ DOMAIN $SecurityContextOf = Requests$
$\land \forall\, r \in Requests :$
$\land\ RequestedTenantOf[r] \in Tenants$
$\land\ AuthenticatedTenantOf[r] \in Tenants$
$\land\ QueryOf[r] \in Queries$
$\land\ ClientOf[r] \in Clients$
$\land\ ProtocolOf[r] \in \{\text{"FLIGHT"}, \text{"REST"}\}$
$\land\ MaxSnapshot \geq 1 \land MaxAttempts \geq 1$
$\land\ MaxCrashes \in Nat \land MaxTimeouts \in Nat$
$\land\ FlightPopulatesCache \in$ BOOLEAN
$\land\ MutantSkipTenantCheck \in$ BOOLEAN
$\land\ Requests \cap Workers = \{\}$

$NONE \triangleq$ "NONE"

$SecCtxs \triangleq \{SecurityContextOf[r] : r \in Requests\}$

$NoEntry \triangleq [valid \mapsto$ FALSE, $blob_id \mapsto NONE]$

$FreeBlob \triangleq [allocated \mapsto$ FALSE,
$staged \mapsto$ FALSE, written by a worker, not yet published
$req \mapsto NONE$, staging request (if staged)
$epoch \mapsto 0$, staging lease epoch (if staged)
$tenant \mapsto NONE$,

$query \mapsto NONE$,
$version \mapsto 0$,
$security_context \mapsto NONE$]

--algorithm *DataLakehouseGateway*

variables

$catalog_snapshot = [t \in Tenants \mapsto 1]$,

$cache = [t \in Tenants \mapsto [q \in Queries \mapsto [v \in 1 \,..\, MaxSnapshot \mapsto$
$[s \in SecCtxs \mapsto NoEntry]]]]$,

$task_queue = \langle\rangle$,

$jobs = [r \in Requests \mapsto$
$[state \mapsto$ "IDLE",
$owner \mapsto NONE$,
$lease_epoch \mapsto 0$,
$attempts \mapsto 0$,
$requested_tenant \mapsto RequestedTenantOf[r]$,
$tenant \mapsto NONE$,
$query \mapsto QueryOf[r]$,
$security_context \mapsto NONE$,
$version \mapsto 0$,
$blob_ptr \mapsto NONE$,
$protocol \mapsto ProtocolOf[r]$,
$rls_applied \mapsto$ FALSE,
$executed_predicate_tenant \mapsto NONE$]],

$blob_storage = [b \in Blobs \mapsto FreeBlob]$,
$available_blobs = Blobs$,

$worker_alive = [w \in Workers \mapsto$ TRUE],

$client_received = [r \in Requests \mapsto$
$[delivered \mapsto$ FALSE,
$released \mapsto$ FALSE, client finished / dropped result
$blob_id \mapsto NONE$,
$tenant \mapsto NONE$,
$query \mapsto NONE$,

$version \mapsto 0,$
$security_context \mapsto NONE]],$

$commit_count = [r \in Requests \mapsto 0],$

$crashes = 0,$ fault budget: worker crashes
$timeouts = 0$ **;** fault budget: lease timeouts on live workers (zombies)

define
$Pending \triangleq \{\text{“QUEUED”}, \text{“CLAIMED”}, \text{“RUNNING”}, \text{“RESULT_WRITTEN”}\}$
$Active \triangleq \{\text{“CLAIMED”}, \text{“RUNNING”}, \text{“RESULT_WRITTEN”}\}$
$Terminal \triangleq \{\text{“COMPLETED”}, \text{“FAILED”}, \text{“UNAUTHORIZED”}\}$
$AllStates \triangleq \{\text{“IDLE”}\} \cup Pending \cup Terminal$

$EffTenant(r) \triangleq$ IF $MutantSkipTenantCheck$
THEN $RequestedTenantOf[r]$
ELSE $AuthenticatedTenantOf[r]$

$HoldsLease(r, e, w, st) \triangleq$
$\land\ jobs[r].lease_epoch = e$
$\land\ jobs[r].owner = w$
$\land\ jobs[r].state = st$

$QCount(r) \triangleq Cardinality(\{i \in 1 \,..\, Len(task_queue) : task_queue[i] = r\})$

$BlobMatches(b, t, q, v, s) \triangleq$
$\land\ blob_storage[b].tenant = t$
$\land\ blob_storage[b].query = q$
$\land\ blob_storage[b].version = v$
$\land\ blob_storage[b].security_context = s$

$BlobCached(b) \triangleq$
$\exists\, t \in Tenants, q \in Queries, v \in 1 \,..\, MaxSnapshot, s \in SecCtxs :$
$cache[t][q][v][s].valid \land cache[t][q][v][s].blob_id = b$

$BlobJobHeld(b) \triangleq$
$\exists\, r \in Requests : jobs[r].blob_ptr = b \land \neg client_received[r].released$

$BlobClientHeld(b) \triangleq$
$\exists\, r \in Requests : client_received[r].delivered$
$\land\ \neg client_received[r].released$
$\land\ client_received[r].blob_id = b$

$StagedOrphan(b) \triangleq$
$jobs[blob_storage[b].req].lease_epoch \neq blob_storage[b].epoch$

$GCable \triangleq \{b \in Blobs :$
$\land\ blob_storage[b].allocated$
$\land$ IF $blob_storage[b].staged$
THEN $StagedOrphan(b)$
ELSE $\neg BlobCached(b) \land \neg BlobJobHeld(b) \land \neg BlobClientHeld(b)\}$

$ExpirableJobs \triangleq \{r \in Requests :$
$\land\ jobs[r].state \in Active$
$\land\ (\neg worker_alive[jobs[r].owner] \lor timeouts < MaxTimeouts)\}$

$CachedKeys \triangleq \{k \in Tenants \times Queries \times (1\,..\,MaxSnapshot) \times SecCtxs :$
$cache[k[1]][k[2]][k[3]][k[4]].valid\}$

$TypeOK \triangleq$
$\land\ catalog_snapshot \in [Tenants \to 1\,..\,MaxSnapshot]$
$\land\ available_blobs \subseteq Blobs$
$\land\ worker_alive \in [Workers \to$ BOOLEAN $]$
$\land\ crashes \in 0\,..\,MaxCrashes$
$\land\ timeouts \in 0\,..\,MaxTimeouts$
$\land\ task_queue \in Seq(Requests)$
$\land \forall\ t \in Tenants,\ q \in Queries,\ v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$\land\ cache[t][q][v][s].valid \in$ BOOLEAN
$\land\ cache[t][q][v][s].blob_id \in Blobs \cup \{NONE\}$
$\land \forall\ b \in Blobs :$
$\land\ blob_storage[b].allocated \in$ BOOLEAN
$\land\ blob_storage[b].staged \in$ BOOLEAN
$\land \forall\ r \in Requests :$
$\land\ jobs[r].state \in AllStates$
$\land\ jobs[r].attempts \in 0\,..\,MaxAttempts$
$\land\ jobs[r].lease_epoch \in Nat$
$\land\ jobs[r].blob_ptr \in Blobs \cup \{NONE\}$
$\land\ commit_count[r] \in 0\,..\,1$
$\land\ client_received[r].delivered \Rightarrow client_received[r].blob_id \neq NONE$

$TenantBinding \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state \notin \{\text{“IDLE”}, \text{“UNAUTHORIZED”}\} \Rightarrow$
$\land\, jobs[r].tenant = AuthenticatedTenantOf[r]$
$\land\, jobs[r].security_context = SecurityContextOf[r]$

$RLSEnforcement \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state \in \{\text{“RUNNING”}, \text{“RESULT_WRITTEN”}, \text{“COMPLETED”}\} \Rightarrow$
$\land\, jobs[r].rls_applied$
$\land\, jobs[r].executed_predicate_tenant = AuthenticatedTenantOf[r]$

$UnauthorizedIsInert \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state = \text{“UNAUTHORIZED”} \Rightarrow$
$\land\, jobs[r].blob_ptr = NONE$
$\land\, jobs[r].owner = NONE$
$\land\, jobs[r].attempts = 0$
$\land\, commit_count[r] = 0$
$\land\, \neg client_received[r].delivered$
$\land\, QCount(r) = 0$

$MismatchRejected \triangleq$
$\forall\, r \in Requests :$
$RequestedTenantOf[r] \neq AuthenticatedTenantOf[r] \Rightarrow$
$\land\, jobs[r].state \in \{\text{“IDLE”}, \text{“UNAUTHORIZED”}\}$
$\land\, \neg client_received[r].delivered$

$TenantIsolation \triangleq$
$\forall\, r \in Requests :$
$client_received[r].delivered \Rightarrow$
$\land\, client_received[r].tenant = AuthenticatedTenantOf[r]$
$\land\, client_received[r].security_context = SecurityContextOf[r]$
$\land\, client_received[r].query = QueryOf[r]$

$DeliveredResultIntegrity \triangleq$
$\forall\, r \in Requests :$
$(client_received[r].delivered \land \neg client_received[r].released) \Rightarrow$
$\land\ blob_storage[client_received[r].blob_id].allocated$
$\land \neg blob_storage[client_received[r].blob_id].staged$
$\land\ BlobMatches(client_received[r].blob_id,$
$AuthenticatedTenantOf[r],\ QueryOf[r],$
$client_received[r].version,\ SecurityContextOf[r])$

$ClaimCheckIntegrity \triangleq$
$\forall\, r \in Requests :$
$(jobs[r].state \in \{\text{“RESULT_WRITTEN”},\ \text{“COMPLETED”}\}$
$\land \neg client_received[r].released) \Rightarrow$
$\land\ blob_storage[jobs[r].blob_ptr].allocated$
$\land \neg blob_storage[jobs[r].blob_ptr].staged$
$\land\ BlobMatches(jobs[r].blob_ptr,\ jobs[r].tenant,\ jobs[r].query,$
$jobs[r].version,\ jobs[r].security_context)$

$BlobPtrIffResultState \triangleq$
$\forall\, r \in Requests :$
$(jobs[r].blob_ptr \neq NONE) \equiv$
$(jobs[r].state \in \{\text{“RESULT_WRITTEN”},\ \text{“COMPLETED”}\})$

$NoSharedBlobPointer \triangleq$
$\forall\, r1,\ r2 \in Requests :$
$(r1 \neq r2 \land jobs[r1].blob_ptr \neq NONE \land jobs[r2].blob_ptr \neq NONE$
$\land \neg client_received[r1].released \land \neg client_received[r2].released) \Rightarrow$
$jobs[r1].blob_ptr \neq jobs[r2].blob_ptr$

$BlobPoolConsistency \triangleq$
$available_blobs = \{b \in Blobs : \neg blob_storage[b].allocated\}$

$StagedBlobWellFormed \triangleq$
$\forall\, b \in Blobs :$
$blob_storage[b].staged \Rightarrow$
$\land\ blob_storage[b].allocated$
$\land\ blob_storage[b].req \in Requests$
$\land\ jobs[blob_storage[b].req].lease_epoch \geq blob_storage[b].epoch$

$CacheIntegrity \triangleq$
$\forall\, t \in Tenants,\ q \in Queries,\ v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[t][q][v][s].valid \Rightarrow$
$\land\ blob_storage[cache[t][q][v][s].blob_id].allocated$

$\land \neg blob_storage[cache[t][q][v][s].blob_id].staged$
$\land BlobMatches(cache[t][q][v][s].blob_id,\ t,\ q,\ v,\ s)$

$CacheFreshness \triangleq$
$\forall\, t \in Tenants,\ q \in Queries,\ v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[t][q][v][s].valid \Rightarrow v = catalog_snapshot[t]$

$ExactlyOnceCommit \triangleq$
$\forall\, r \in Requests :$
$\land\ commit_count[r] \leq 1$
$\land\ (jobs[r].state = \text{“COMPLETED”}) \equiv (commit_count[r] = 1)$

$FailedImpliesExhausted \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state = \text{“FAILED”} \Rightarrow jobs[r].attempts = MaxAttempts$

$LeaseOwnership \triangleq$
$\forall\, r \in Requests :$
$\land\ jobs[r].state \in Active \Rightarrow jobs[r].owner \in Workers$
$\land\ jobs[r].state \notin Active \Rightarrow jobs[r].owner = NONE$

$QueueConsistency \triangleq$
$\forall\, r \in Requests :$
$\land\ jobs[r].state = \text{“QUEUED”} \Rightarrow QCount(r) = 1$
$\land\ jobs[r].state \neq \text{“QUEUED”} \Rightarrow QCount(r) = 0$

$FlightIsLocal \triangleq$
$\forall\, r \in Requests :$
$ProtocolOf[r] = \text{“FLIGHT”} \Rightarrow$
$\land\ jobs[r].state \in \{\text{“IDLE”},\ \text{“COMPLETED”},\ \text{“UNAUTHORIZED”}\}$
$\land\ jobs[r].owner = NONE$
$\land\ QCount(r) = 0$

$SnapshotBound \triangleq$
$\forall\, r \in Requests :$
$\land\ jobs[r].state \notin \{\text{“IDLE”},\ \text{“UNAUTHORIZED”}\} \Rightarrow$
$jobs[r].version \in 1\,..\,catalog_snapshot[jobs[r].tenant]$
$\land\ client_received[r].delivered \Rightarrow$

client_received[*r*].*version*
∈ 1 .. *catalog_snapshot*[*AuthenticatedTenantOf*[*r*]]

end define ;

fair process *ClientProcess* ∈ *Requests*
variables *req_version* = 0 **;**
begin
Start:

if ¬*MutantSkipTenantCheck*
∧ *AuthenticatedTenantOf*[*self*] ≠ *RequestedTenantOf*[*self*] **then**
jobs[*self*] := [*jobs*[*self*] EXCEPT !.*state* = "UNAUTHORIZED"] **;**
goto *Release* **;**
end if ;

BindSnapshot:
req_version := *catalog_snapshot*[*EffTenant*(*self*)] **;**

CheckCache:
if *cache*[*EffTenant*(*self*)][*QueryOf*[*self*]][*req_version*][*SecurityContextOf*[*self*]].*valid* **then**
client_received[*self*] :=
[*client_received*[*self*] EXCEPT
!.*delivered* = TRUE,
!.*blob_id* = *cache*[*EffTenant*(*self*)][*QueryOf*[*self*]][*req_version*][*SecurityContextOf*[*self*]].*blob_id*,
!.*tenant* = *EffTenant*(*self*),
!.*query* = *QueryOf*[*self*],
!.*version* = *req_version*,
!.*security_context* = *SecurityContextOf*[*self*]] **;**
goto *Release* **;**
end if ;

InitializeJob:
jobs[*self*] := [*jobs*[*self*] EXCEPT
!.*tenant* = *EffTenant*(*self*),
!.*security_context* = *SecurityContextOf*[*self*],
!.*version* = *req_version*] **;**

Route:

if $ProtocolOf[self] =$ "FLIGHT" **then**
goto *ExecuteFlight* ;
else
goto *EnqueueREST* ;
end if ;

ExecuteFlight:

await $available_blobs \neq \{\}$;
with $b \in available_blobs$ **do**
$available_blobs := available_blobs \setminus \{b\}$;
$blob_storage[b] := [FreeBlob$ EXCEPT
$!.allocated =$ TRUE,
$!.tenant = jobs[self].tenant$,
$!.query = jobs[self].query$,
$!.version = jobs[self].version$,
$!.security_context = jobs[self].security_context]$;
$jobs[self] := [jobs[self]$ EXCEPT
$!.state =$ "COMPLETED",
$!.lease_epoch = 1$,
$!.attempts = 1$,
$!.blob_ptr = b$,
$!.rls_applied =$ TRUE,
$!.executed_predicate_tenant = jobs[self].tenant]$;
$commit_count[self] := commit_count[self] + 1$;
$client_received[self] := [client_received[self]$ EXCEPT
$!.delivered =$ TRUE,
$!.blob_id = b$,
$!.tenant = jobs[self].tenant$,
$!.query = jobs[self].query$,
$!.version = jobs[self].version$,
$!.security_context = jobs[self].security_context]$;
if *FlightPopulatesCache*
$\wedge\, jobs[self].version = catalog_snapshot[jobs[self].tenant]$ **then**
$cache[jobs[self].tenant][jobs[self].query][jobs[self].version][jobs[self].security_context] :=$
$[valid \mapsto$ TRUE, $blob_id \mapsto b]$;
end if ;
end with ;
goto *Release* ;

EnqueueREST:
$jobs[self] := [jobs[self]$ EXCEPT $!.state =$ "QUEUED", $!.attempts = @ + 1]$;
$task_queue := Append(task_queue, self)$;

AwaitREST:
await $jobs[self].state \in Terminal$;
if $jobs[self].state =$ "COMPLETED" **then**
$client_received[self] := [client_received[self]$ EXCEPT
$!.delivered =$ TRUE,
$!.blob_id = jobs[self].blob_ptr$,
$!.tenant = jobs[self].tenant$,
$!.query = jobs[self].query$,
$!.version = jobs[self].version$,
$!.security_context = jobs[self].security_context]$;
end if ;

Release:

$client_received[self] := [client_received[self]$ EXCEPT $!.released =$ TRUE] ;
end process ;

fair process $ComputeWorker \in Workers$
variables $curr_req = NONE$, $curr_epoch = 0$, $curr_blob = NONE$;
begin
ClaimTask:
while TRUE **do**
await $Len(task_queue) > 0 \land worker_alive[self]$;
with $r = Head(task_queue)$ **do**
$task_queue := Tail(task_queue)$;
$curr_req := r$;
$curr_blob := NONE$;
if $jobs[r].state =$ "QUEUED" **then**
$curr_epoch := jobs[r].lease_epoch + 1$;
$jobs[r] := [jobs[r]$ EXCEPT
$!.owner = self$, $!.state =$ "CLAIMED", $!.lease_epoch = @ + 1]$;
else
$curr_epoch := 0$;
end if ;
end with ;

StartExecution:
if $curr_epoch > 0 \land worker_alive[self]$
$\land\ HoldsLease(curr_req, curr_epoch, self,$ "CLAIMED") **then**

$jobs[curr_req] := [jobs[curr_req]$ EXCEPT
$!.state =$ "RUNNING",
$!.rls_applied =$ TRUE,
$!.executed_predicate_tenant = jobs[curr_req].tenant]$;
else
goto *ClaimTask* ;
end if ;

AllocateBlob:

await $\neg worker_alive[self]$
$\lor \neg HoldsLease(curr_req, curr_epoch, self,$ "RUNNING")
$\lor available_blobs \neq \{\}$;
if $worker_alive[self]$
$\land HoldsLease(curr_req, curr_epoch, self,$ "RUNNING")
$\land available_blobs \neq \{\}$ **then**
with $b \in available_blobs$ **do**
$available_blobs := available_blobs \setminus \{b\}$;
$curr_blob := b$;
$blob_storage[b] := [FreeBlob$ EXCEPT
$!.allocated =$ TRUE,
$!.staged =$ TRUE,
$!.req = curr_req$,
$!.epoch = curr_epoch$,
$!.tenant = jobs[curr_req].tenant$,
$!.query = jobs[curr_req].query$,
$!.version = jobs[curr_req].version$,
$!.security_context = jobs[curr_req].security_context]$;
end with ;
else
goto *ClaimTask* ; a staged blob (if any) is left to the *GC*
end if ;

PublishPointer:
if $worker_alive[self]$
$\land HoldsLease(curr_req, curr_epoch, self,$ "RUNNING") **then**
$jobs[curr_req] := [jobs[curr_req]$ EXCEPT
$!.blob_ptr = curr_blob$, $!.state =$ "RESULT_WRITTEN"] ;
$blob_storage[curr_blob] := [blob_storage[curr_blob]$ EXCEPT $!.staged =$ FALSE] ;
else
goto *ClaimTask* ; fenced: blob is an orphan, *GC* reclaims it
end if ;

CommitJob:

if *worker_alive*[*self*]
$\land$ *HoldsLease*(*curr_req*, *curr_epoch*, *self*, "RESULT_WRITTEN") **then**
jobs[*curr_req*] := [*jobs*[*curr_req*] EXCEPT
!.*state* = "COMPLETED", !.*owner* = *NONE*] ;
commit_count[*curr_req*] := *commit_count*[*curr_req*] + 1 ;
else
goto *ClaimTask* ;
end if ;

PopulateCache:

if *worker_alive*[*self*]
$\land$ *jobs*[*curr_req*].*state* = "COMPLETED"
$\land$ *jobs*[*curr_req*].*blob_ptr* = *curr_blob*
$\land$ *blob_storage*[*curr_blob*].*allocated*
$\land$ $\neg$*blob_storage*[*curr_blob*].*staged*
$\land$ *BlobMatches*(*curr_blob*, *jobs*[*curr_req*].*tenant*, *jobs*[*curr_req*].*query*,
jobs[*curr_req*].*version*, *jobs*[*curr_req*].*security_context*)
$\land$ *jobs*[*curr_req*].*version* = *catalog_snapshot*[*jobs*[*curr_req*].*tenant*] **then**
cache[*jobs*[*curr_req*].*tenant*][*jobs*[*curr_req*].*query*][*jobs*[*curr_req*].*version*][*jobs*[*curr_req*].*security_context*] :=
[*valid* $\mapsto$ TRUE, *blob_id* $\mapsto$ *curr_blob*] ;
end if ;
end while ;
end process ;

fair process *LeaseManager* = "LeaseManager"
begin
LeaseLoop:
while TRUE **do**
with $r \in$ *ExpirableJobs* **do**

if *worker_alive*[*jobs*[*r*].*owner*] **then**
timeouts := *timeouts* + 1 ;
end if ;
if *jobs*[*r*].*attempts* < *MaxAttempts* **then**
jobs[*r*] := [*jobs*[*r*] EXCEPT

```
!.owner = NONE, !.state = "QUEUED",
!.attempts = @ + 1, !.lease_epoch = @ + 1,
!.blob_ptr = NONE, !.rls_applied = FALSE,
!.executed_predicate_tenant = NONE] ;
task_queue := Append(task_queue, r) ;
else
jobs[r] := [jobs[r] EXCEPT
!.owner = NONE, !.state = "FAILED",
!.lease_epoch = @ + 1, !.blob_ptr = NONE,
!.rls_applied = FALSE,
!.executed_predicate_tenant = NONE] ;
end if ;
end with ;
end while ;
end process ;
```

```
fair process GarbageCollector = "GarbageCollector"
begin
GCLoop:
while TRUE do
with b ∈ GCable do
blob_storage[b] := FreeBlob ;
available_blobs := available_blobs ∪ {b} ;
end with ;
end while ;
end process ;
```

```
fair process CacheEvictor = "CacheEvictor"
begin
EvictLoop:
while TRUE do
with e ∈ CachedKeys do
cache[e[1]][e[2]][e[3]][e[4]] := NoEntry ;
end with ;
end while ;
```

end process ;

process *Crasher* = "Crasher"
begin
CrashLoop:
while $crashes < MaxCrashes$ **do**
with $w \in \{x \in Workers : worker_alive[x]\}$ **do**
$worker_alive[w] :=$ FALSE **;**
$crashes := crashes + 1$ **;**
end with ;
end while ;
end process ;

fair process *Recoverer* = "Recoverer"
begin
RecoverLoop:
while TRUE **do**
with $w \in \{x \in Workers :$
$\land \neg worker_alive[x]$
$\land \forall\, r \in Requests :$
$\neg(jobs[r].owner = x \land jobs[r].state \in Active)\}$ **do**
$worker_alive[w] :=$ TRUE **;**
end with ;
end while ;
end process ;

fair process *LakehouseETL* = "LakehouseETL"
begin
ETLCommit:
while TRUE **do**
with $t \in \{x \in Tenants : catalog_snapshot[x] < MaxSnapshot\}$ **do**

$cache[t] := [q \in Queries \mapsto [v \in 1 \,..\, MaxSnapshot \mapsto [s \in SecCtxs \mapsto$
IF $v < catalog_snapshot[t] + 1$
THEN $NoEntry$
ELSE $cache[t][q][v][s]]]]$;
$catalog_snapshot[t] := catalog_snapshot[t] + 1$;
end with ;
end while ;
end process ;

end algorithm ;

VARIABLES pc, $catalog_snapshot$, $cache$, $task_queue$, $jobs$, $blob_storage$, $available_blobs$, $worker_alive$, $client_received$, $commit_count$, $crashes$, $timeouts$

$Pending \triangleq \{$"QUEUED", "CLAIMED", "RUNNING", "RESULT_WRITTEN"$\}$
$Active \triangleq \{$"CLAIMED", "RUNNING", "RESULT_WRITTEN"$\}$
$Terminal \triangleq \{$"COMPLETED", "FAILED", "UNAUTHORIZED"$\}$
$AllStates \triangleq \{$"IDLE"$\} \cup Pending \cup Terminal$

$EffTenant(r) \triangleq$ IF $MutantSkipTenantCheck$
THEN $RequestedTenantOf[r]$
ELSE $AuthenticatedTenantOf[r]$

$HoldsLease(r, e, w, st) \triangleq$
$\land\ jobs[r].lease_epoch = e$
$\land\ jobs[r].owner = w$
$\land\ jobs[r].state = st$

$QCount(r) \triangleq Cardinality(\{i \in 1 \,..\, Len(task_queue) : task_queue[i] = r\})$

$BlobMatches(b, t, q, v, s) \triangleq$
$\land\ blob_storage[b].tenant = t$
$\land\ blob_storage[b].query = q$
$\land\ blob_storage[b].version = v$
$\land\ blob_storage[b].security_context = s$

$BlobCached(b) \triangleq$
$\exists\, t \in Tenants, q \in Queries, v \in 1 \,..\, MaxSnapshot, s \in SecCtxs :$
$cache[t][q][v][s].valid \land cache[t][q][v][s].blob_id = b$

$BlobJobHeld(b) \triangleq$
$\exists\, r \in Requests : jobs[r].blob_ptr = b \land \neg client_received[r].released$

$BlobClientHeld(b) \triangleq$
$\exists\, r \in Requests : client_received[r].delivered$
$\land \neg client_received[r].released$
$\land client_received[r].blob_id = b$

$StagedOrphan(b) \triangleq$
$jobs[blob_storage[b].req].lease_epoch \neq blob_storage[b].epoch$

$GCable \triangleq \{b \in Blobs :$
$\land\ blob_storage[b].allocated$
$\land$ IF $blob_storage[b].staged$
THEN $StagedOrphan(b)$
ELSE $\neg BlobCached(b) \land \neg BlobJobHeld(b) \land \neg BlobClientHeld(b)\}$

$ExpirableJobs \triangleq \{r \in Requests :$
$\land\ jobs[r].state \in Active$
$\land\ (\neg worker_alive[jobs[r].owner] \lor timeouts < MaxTimeouts)\}$

$CachedKeys \triangleq \{k \in Tenants \times Queries \times (1 \,..\, MaxSnapshot) \times SecCtxs :$
$cache[k[1]][k[2]][k[3]][k[4]].valid\}$

$TypeOK \triangleq$
$\land\ catalog_snapshot \in [Tenants \to 1 \,..\, MaxSnapshot]$
$\land\ available_blobs \subseteq Blobs$
$\land\ worker_alive \in [Workers \to$ BOOLEAN $]$
$\land\ crashes \in 0 \,..\, MaxCrashes$
$\land\ timeouts \in 0 \,..\, MaxTimeouts$
$\land\ task_queue \in Seq(Requests)$
$\land \forall\, t \in Tenants,\ q \in Queries,\ v \in 1 \,..\, MaxSnapshot,\ s \in SecCtxs :$
$\land\ cache[t][q][v][s].valid \in$ BOOLEAN
$\land\ cache[t][q][v][s].blob_id \in Blobs \cup \{NONE\}$
$\land \forall\, b \in Blobs :$
$\land\ blob_storage[b].allocated \in$ BOOLEAN
$\land\ blob_storage[b].staged \in$ BOOLEAN
$\land \forall\, r \in Requests :$
$\land\ jobs[r].state \in AllStates$
$\land\ jobs[r].attempts \in 0 \,..\, MaxAttempts$

$\land\ jobs[r].lease_epoch \in Nat$
$\land\ jobs[r].blob_ptr \in Blobs \cup \{NONE\}$
$\land\ commit_count[r] \in 0 \,..\, 1$
$\land\ client_received[r].delivered \Rightarrow client_received[r].blob_id \neq NONE$

$TenantBinding \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state \notin \{\text{“IDLE”},\ \text{“UNAUTHORIZED”}\} \Rightarrow$
$\land\ jobs[r].tenant = AuthenticatedTenantOf[r]$
$\land\ jobs[r].security_context = SecurityContextOf[r]$

$RLSEnforcement \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state \in \{\text{“RUNNING”},\ \text{“RESULT_WRITTEN”},\ \text{“COMPLETED”}\} \Rightarrow$
$\land\ jobs[r].rls_applied$
$\land\ jobs[r].executed_predicate_tenant = AuthenticatedTenantOf[r]$

$UnauthorizedIsInert \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state = \text{“UNAUTHORIZED”} \Rightarrow$
$\land\ jobs[r].blob_ptr = NONE$
$\land\ jobs[r].owner = NONE$
$\land\ jobs[r].attempts = 0$
$\land\ commit_count[r] = 0$
$\land\ \neg client_received[r].delivered$
$\land\ QCount(r) = 0$

$MismatchRejected \triangleq$
$\forall\, r \in Requests :$
$RequestedTenantOf[r] \neq AuthenticatedTenantOf[r] \Rightarrow$
$\land\ jobs[r].state \in \{\text{“IDLE”},\ \text{“UNAUTHORIZED”}\}$
$\land\ \neg client_received[r].delivered$

$TenantIsolation \triangleq$
$\forall\, r \in Requests :$
$client_received[r].delivered \Rightarrow$
$\land\ client_received[r].tenant = AuthenticatedTenantOf[r]$
$\land\ client_received[r].security_context = SecurityContextOf[r]$

$\land\ client_received[r].query = QueryOf[r]$

$DeliveredResultIntegrity \triangleq$
$\forall\, r \in Requests :$
$(client_received[r].delivered \land \neg client_received[r].released) \Rightarrow$
$\land\ blob_storage[client_received[r].blob_id].allocated$
$\land \neg blob_storage[client_received[r].blob_id].staged$
$\land\ BlobMatches(client_received[r].blob_id,$
$AuthenticatedTenantOf[r],\ QueryOf[r],$
$client_received[r].version,\ SecurityContextOf[r])$

$ClaimCheckIntegrity \triangleq$
$\forall\, r \in Requests :$
$(jobs[r].state \in \{\text{“RESULT_WRITTEN”},\ \text{“COMPLETED”}\}$
$\land \neg client_received[r].released) \Rightarrow$
$\land\ blob_storage[jobs[r].blob_ptr].allocated$
$\land \neg blob_storage[jobs[r].blob_ptr].staged$
$\land\ BlobMatches(jobs[r].blob_ptr,\ jobs[r].tenant,\ jobs[r].query,$
$jobs[r].version,\ jobs[r].security_context)$

$BlobPtrIffResultState \triangleq$
$\forall\, r \in Requests :$
$(jobs[r].blob_ptr \neq NONE) \equiv$
$(jobs[r].state \in \{\text{“RESULT_WRITTEN”},\ \text{“COMPLETED”}\})$

$NoSharedBlobPointer \triangleq$
$\forall\, r1,\ r2 \in Requests :$
$(r1 \neq r2 \land jobs[r1].blob_ptr \neq NONE \land jobs[r2].blob_ptr \neq NONE$
$\land \neg client_received[r1].released \land \neg client_received[r2].released) \Rightarrow$
$jobs[r1].blob_ptr \neq jobs[r2].blob_ptr$

$BlobPoolConsistency \triangleq$
$available_blobs = \{b \in Blobs : \neg blob_storage[b].allocated\}$

$StagedBlobWellFormed \triangleq$
$\forall\, b \in Blobs :$
$blob_storage[b].staged \Rightarrow$
$\land\ blob_storage[b].allocated$
$\land\ blob_storage[b].req \in Requests$

$\land\ jobs[blob_storage[b].req].lease_epoch \geq blob_storage[b].epoch$

$CacheIntegrity \triangleq$
$\forall\, t \in Tenants,\ q \in Queries,\ v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[t][q][v][s].valid \Rightarrow$
$\land\ blob_storage[cache[t][q][v][s].blob_id].allocated$
$\land\ \neg blob_storage[cache[t][q][v][s].blob_id].staged$
$\land\ BlobMatches(cache[t][q][v][s].blob_id,\ t,\ q,\ v,\ s)$

$CacheFreshness \triangleq$
$\forall\, t \in Tenants,\ q \in Queries,\ v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[t][q][v][s].valid \Rightarrow v = catalog_snapshot[t]$

$ExactlyOnceCommit \triangleq$
$\forall\, r \in Requests :$
$\land\ commit_count[r] \leq 1$
$\land\ (jobs[r].state = \text{“COMPLETED”}) \equiv (commit_count[r] = 1)$

$FailedImpliesExhausted \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state = \text{“FAILED”} \Rightarrow jobs[r].attempts = MaxAttempts$

$LeaseOwnership \triangleq$
$\forall\, r \in Requests :$
$\land\ jobs[r].state \in Active \Rightarrow jobs[r].owner \in Workers$
$\land\ jobs[r].state \notin Active \Rightarrow jobs[r].owner = NONE$

$QueueConsistency \triangleq$
$\forall\, r \in Requests :$
$\land\ jobs[r].state = \text{“QUEUED”} \Rightarrow QCount(r) = 1$
$\land\ jobs[r].state \neq \text{“QUEUED”} \Rightarrow QCount(r) = 0$

$FlightIsLocal \triangleq$
$\forall\, r \in Requests :$
$ProtocolOf[r] = \text{“FLIGHT”} \Rightarrow$
$\land\ jobs[r].state \in \{\text{“IDLE”},\ \text{“COMPLETED”},\ \text{“UNAUTHORIZED”}\}$
$\land\ jobs[r].owner = NONE$

$\land\ QCount(r) = 0$

$SnapshotBound\ \triangleq$
$\forall\, r \in Requests :$
$\land\ jobs[r].state \notin \{\text{“IDLE”},\ \text{“UNAUTHORIZED”}\} \Rightarrow$
$jobs[r].version \in 1\ ..\ catalog_snapshot[jobs[r].tenant]$
$\land\ client_received[r].delivered \Rightarrow$
$client_received[r].version$
$\in 1\ ..\ catalog_snapshot[AuthenticatedTenantOf[r]]$

VARIABLES $req_version,\ curr_req,\ curr_epoch,\ curr_blob$

$vars\ \triangleq\ \langle pc,\ catalog_snapshot,\ cache,\ task_queue,\ jobs,\ blob_storage,$
$available_blobs,\ worker_alive,\ client_received,\ commit_count,$
$crashes,\ timeouts,\ req_version,\ curr_req,\ curr_epoch,\ curr_blob\rangle$

$ProcSet\ \triangleq\ (Requests) \cup (Workers) \cup \{\text{“LeaseManager”}\} \cup \{\text{“GarbageCollector”}\} \cup \{\text{“CacheEvictor”}\} \cup \{\text{“Crashe}$

$Init\ \triangleq$ Global variables
$\land\ catalog_snapshot = [t \in Tenants \mapsto 1]$
$\land\ cache = [t \in Tenants \mapsto [q \in Queries \mapsto [v \in 1\ ..\ MaxSnapshot \mapsto$
$[s \in SecCtxs \mapsto NoEntry]]]]$
$\land\ task_queue = \langle\rangle$
$\land\ jobs = [r \in Requests \mapsto$
$[state \mapsto \text{“IDLE”},$
$owner \mapsto NONE,$
$lease_epoch \mapsto 0,$
$attempts \mapsto 0,$
$requested_tenant \mapsto RequestedTenantOf[r],$
$tenant \mapsto NONE,$
$query \mapsto QueryOf[r],$
$security_context \mapsto NONE,$
$version \mapsto 0,$
$blob_ptr \mapsto NONE,$
$protocol \mapsto ProtocolOf[r],$
$rls_applied \mapsto \text{FALSE},$
$executed_predicate_tenant \mapsto NONE]]$
$\land\ blob_storage = [b \in Blobs \mapsto FreeBlob]$
$\land\ available_blobs = Blobs$
$\land\ worker_alive = [w \in Workers \mapsto \text{TRUE}]$
$\land\ client_received = [r \in Requests \mapsto$
$[delivered \mapsto \text{FALSE},$
$released \mapsto \text{FALSE},$
$blob_id \mapsto NONE,$
$tenant \mapsto NONE,$

$query \mapsto NONE$,
$version \mapsto 0$,
$security_context \mapsto NONE$]]
$\land\ commit_count = [r \in Requests \mapsto 0]$
$\land\ crashes = 0$
$\land\ timeouts = 0$

$\land\ req_version = [self \in Requests \mapsto 0]$

$\land\ curr_req = [self \in Workers \mapsto NONE]$
$\land\ curr_epoch = [self \in Workers \mapsto 0]$
$\land\ curr_blob = [self \in Workers \mapsto NONE]$
$\land\ pc = [self \in ProcSet \mapsto \text{CASE } self \in Requests \to$ "Start"
$\square self \in Workers \to$ "ClaimTask"
$\square self =$ "LeaseManager" $\to$ "LeaseLoop"
$\square self =$ "GarbageCollector" $\to$ "GCLoop"
$\square self =$ "CacheEvictor" $\to$ "EvictLoop"
$\square self =$ "Crasher" $\to$ "CrashLoop"
$\square self =$ "Recoverer" $\to$ "RecoverLoop"
$\square self =$ "LakehouseETL" $\to$ "ETLCommit"]

$Start(self) \triangleq\ \land\ pc[self] =$ "Start"
$\land$ IF $\neg MutantSkipTenantCheck$
$\land\ AuthenticatedTenantOf[self] \neq RequestedTenantOf[self]$
THEN $\land\ jobs' = [jobs$ EXCEPT $![self] = [jobs[self]$ EXCEPT $!.state =$ "UNAUTHORIZED"]]
$\land\ pc' = [pc$ EXCEPT $![self] =$ "Release"]
ELSE $\land\ pc' = [pc$ EXCEPT $![self] =$ "BindSnapshot"]
$\land\ jobs' = jobs$
$\land$ UNCHANGED $\langle catalog_snapshot, cache, task_queue,$
$blob_storage, available_blobs, worker_alive,$
$client_received, commit_count, crashes,$
$timeouts, req_version, curr_req, curr_epoch,$
$curr_blob\rangle$

$BindSnapshot(self) \triangleq\ \land\ pc[self] =$ "BindSnapshot"
$\land\ req_version' = [req_version$ EXCEPT $![self] = catalog_snapshot[EffTenant(self)]]$
$\land\ pc' = [pc$ EXCEPT $![self] =$ "CheckCache"]
$\land$ UNCHANGED $\langle catalog_snapshot, cache, task_queue,$
$jobs, blob_storage, available_blobs,$
$worker_alive, client_received,$
$commit_count, crashes, timeouts,$
$curr_req, curr_epoch, curr_blob\rangle$

$CheckCache(self) \triangleq\ \land\ pc[self] =$ "CheckCache"
$\land$ IF $cache[EffTenant(self)][QueryOf[self]][req_version[self]][SecurityContextOf[self]].valid$

THEN $\land$ $client_received' = [client_received$ EXCEPT $![self] = [client_received[self]$ EXCEPT
$!.delivered =$ TRUE,
$!.blob_id = cache[EffTenant(self)][QueryOf[self]][req_version[self]][SecurityContextOf[self]].blob_id$,
$!.tenant = EffTenant(self)$,
$!.query = QueryOf[self]$,
$!.version = req_version[self]$,
$!.security_context = SecurityContextOf[self]]]$
$\land$ $pc' = [pc$ EXCEPT $![self] =$ "Release"$]$
ELSE $\land$ $pc' = [pc$ EXCEPT $![self] =$ "InitializeJob"$]$
$\land$ UNCHANGED $client_received$
$\land$ UNCHANGED $\langle catalog_snapshot,\ cache,\ task_queue,\ jobs,$
$blob_storage,\ available_blobs,$
$worker_alive,\ commit_count,\ crashes,$
$timeouts,\ req_version,\ curr_req,$
$curr_epoch,\ curr_blob\rangle$

$InitializeJob(self) \triangleq$ $\land$ $pc[self] =$ "InitializeJob"
$\land$ $jobs' = [jobs$ EXCEPT $![self] = [jobs[self]$ EXCEPT
$!.tenant = EffTenant(self)$,
$!.security_context = SecurityContextOf[self]$,
$!.version = req_version[self]]]$
$\land$ $pc' = [pc$ EXCEPT $![self] =$ "Route"$]$
$\land$ UNCHANGED $\langle catalog_snapshot,\ cache,\ task_queue,$
$blob_storage,\ available_blobs,$
$worker_alive,\ client_received,$
$commit_count,\ crashes,\ timeouts,$
$req_version,\ curr_req,\ curr_epoch,$
$curr_blob\rangle$

$Route(self) \triangleq$ $\land$ $pc[self] =$ "Route"
$\land$ IF $ProtocolOf[self] =$ "FLIGHT"
THEN $\land$ $pc' = [pc$ EXCEPT $![self] =$ "ExecuteFlight"$]$
ELSE $\land$ $pc' = [pc$ EXCEPT $![self] =$ "EnqueueREST"$]$
$\land$ UNCHANGED $\langle catalog_snapshot,\ cache,\ task_queue,\ jobs,$
$blob_storage,\ available_blobs,\ worker_alive,$
$client_received,\ commit_count,\ crashes,$
$timeouts,\ req_version,\ curr_req,\ curr_epoch,$
$curr_blob\rangle$

$ExecuteFlight(self) \triangleq$ $\land$ $pc[self] =$ "ExecuteFlight"
$\land$ $available_blobs \neq \{\}$
$\land \exists\, b \in available_blobs :$
$\land$ $available_blobs' = available_blobs \setminus \{b\}$
$\land$ $blob_storage' = [blob_storage$ EXCEPT $![b] = [FreeBlob$ EXCEPT
$!.allocated =$ TRUE,

$!.tenant = jobs[self].tenant,$
$!.query = jobs[self].query,$
$!.version = jobs[self].version,$
$!.security_context = jobs[self].security_context]]$
$\land\ jobs' = [jobs \text{ EXCEPT } ![self] = [jobs[self] \text{ EXCEPT}$
$!.state = \text{“COMPLETED”},$
$!.lease_epoch = 1,$
$!.attempts = 1,$
$!.blob_ptr = b,$
$!.rls_applied = \text{TRUE},$
$!.executed_predicate_tenant = jobs[self].tenant]]$
$\land\ commit_count' = [commit_count \text{ EXCEPT } ![self] = commit_count[self] + 1]$
$\land\ client_received' = [client_received \text{ EXCEPT } ![self] = [client_received[self] \text{ EXCEPT}$
$!.delivered = \text{TRUE},$
$!.blob_id = b,$
$!.tenant = jobs'[self].tenant,$
$!.query = jobs'[self].query,$
$!.version = jobs'[self].version,$
$!.security_context = jobs'[self].security_context]]$
$\land \text{ IF } FlightPopulatesCache$
$\land\ jobs'[self].version = catalog_snapshot[jobs'[self].tenant]$
$\text{ THEN } \land\ cache' = [cache \text{ EXCEPT } ![jobs'[self].tenant][jobs'[self].query][jobs'[self].version][jobs'[self].security_$
$\text{ ELSE } \land \text{TRUE}$
$\land\ cache' = cache$
$\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“Release”}]$
$\land \text{UNCHANGED } \langle catalog_snapshot, task_queue,$
$worker_alive, crashes, timeouts,$
$req_version, curr_req, curr_epoch,$
$curr_blob\rangle$

$EnqueueREST(self) \triangleq \land\ pc[self] = \text{“EnqueueREST”}$
$\land\ jobs' = [jobs \text{ EXCEPT } ![self] = [jobs[self] \text{ EXCEPT } !.state = \text{“QUEUED”}, !.attempts = @ + 1]]$
$\land\ task_queue' = Append(task_queue, self)$
$\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“AwaitREST”}]$
$\land \text{UNCHANGED } \langle catalog_snapshot, cache, blob_storage,$
$available_blobs, worker_alive,$
$client_received, commit_count, crashes,$
$timeouts, req_version, curr_req,$
$curr_epoch, curr_blob\rangle$

$AwaitREST(self) \triangleq \land\ pc[self] = \text{“AwaitREST”}$
$\land\ jobs[self].state \in Terminal$
$\land \text{ IF } jobs[self].state = \text{“COMPLETED”}$
$\text{ THEN } \land\ client_received' = [client_received \text{ EXCEPT } ![self] = [client_received[self] \text{ EXCEPT}$
$!.delivered = \text{TRUE},$

$!.blob_id = jobs[self].blob_ptr,$
$!.tenant = jobs[self].tenant,$
$!.query = jobs[self].query,$
$!.version = jobs[self].version,$
$!.security_context = jobs[self].security_context]]$
ELSE $\land$ TRUE
$\land$ UNCHANGED $client_received$
$\land pc' = [pc$ EXCEPT $![self] =$ "Release"$]$
$\land$ UNCHANGED $\langle catalog_snapshot, cache, task_queue, jobs,$
$blob_storage, available_blobs, worker_alive,$
$commit_count, crashes, timeouts,$
$req_version, curr_req, curr_epoch,$
$curr_blob\rangle$

$Release(self) \triangleq \land pc[self] =$ "Release"
$\land client_received' = [client_received$ EXCEPT $![self] = [client_received[self]$ EXCEPT $!.released =$ TRUE$]]$
$\land pc' = [pc$ EXCEPT $![self] =$ "Done"$]$
$\land$ UNCHANGED $\langle catalog_snapshot, cache, task_queue, jobs,$
$blob_storage, available_blobs, worker_alive,$
$commit_count, crashes, timeouts, req_version,$
$curr_req, curr_epoch, curr_blob\rangle$

$ClientProcess(self) \triangleq Start(self) \lor BindSnapshot(self)$
$\lor CheckCache(self) \lor InitializeJob(self)$
$\lor Route(self) \lor ExecuteFlight(self)$
$\lor EnqueueREST(self) \lor AwaitREST(self)$
$\lor Release(self)$

$ClaimTask(self) \triangleq \land pc[self] =$ "ClaimTask"
$\land Len(task_queue) > 0 \land worker_alive[self]$
$\land$ LET $r \triangleq Head(task_queue)$IN
$\land task_queue' = Tail(task_queue)$
$\land curr_req' = [curr_req$ EXCEPT $![self] = r]$
$\land curr_blob' = [curr_blob$ EXCEPT $![self] = NONE]$
$\land$ IF $jobs[r].state =$ "QUEUED"
THEN $\land curr_epoch' = [curr_epoch$ EXCEPT $![self] = jobs[r].lease_epoch + 1]$
$\land jobs' = [jobs$ EXCEPT $![r] = [jobs[r]$ EXCEPT
$!.owner = self,$ $!.state =$ "CLAIMED", $!.lease_epoch = @ + 1]]$
ELSE $\land curr_epoch' = [curr_epoch$ EXCEPT $![self] = 0]$
$\land jobs' = jobs$
$\land pc' = [pc$ EXCEPT $![self] =$ "StartExecution"$]$
$\land$ UNCHANGED $\langle catalog_snapshot, cache, blob_storage,$
$available_blobs, worker_alive,$
$client_received, commit_count, crashes,$
$timeouts, req_version\rangle$

$StartExecution(self) \triangleq \ \land pc[self] = \text{“StartExecution”}$
$\land$ IF $curr_epoch[self] > 0 \land worker_alive[self]$
$\land HoldsLease(curr_req[self],\ curr_epoch[self],\ self,\ \text{“CLAIMED”})$
THEN $\land jobs' = [jobs$ EXCEPT $![curr_req[self]] = [jobs[curr_req[self]]$ EXCEPT
$!.state = \text{“RUNNING”},$
$!.rls_applied =$ TRUE,
$!.executed_predicate_tenant = jobs[curr_req[self]].tenant]$
$\land pc' = [pc$ EXCEPT $![self] = \text{“AllocateBlob”}]$
ELSE $\land pc' = [pc$ EXCEPT $![self] = \text{“ClaimTask”}]$
$\land jobs' = jobs$
$\land$ UNCHANGED $\langle catalog_snapshot,\ cache,\ task_queue,$
$blob_storage,\ available_blobs,$
$worker_alive,\ client_received,$
$commit_count,\ crashes,\ timeouts,$
$req_version,\ curr_req,\ curr_epoch,$
$curr_blob\rangle$

$AllocateBlob(self) \triangleq \ \land pc[self] = \text{“AllocateBlob”}$
$\land \neg worker_alive[self]$
$\lor \neg HoldsLease(curr_req[self],\ curr_epoch[self],\ self,\ \text{“RUNNING”})$
$\lor available_blobs \neq \{\}$
$\land$ IF $worker_alive[self]$
$\land HoldsLease(curr_req[self],\ curr_epoch[self],\ self,\ \text{“RUNNING”})$
$\land available_blobs \neq \{\}$
THEN $\land \exists\, b \in available_blobs :$
$\land available_blobs' = available_blobs \setminus \{b\}$
$\land curr_blob' = [curr_blob$ EXCEPT $![self] = b]$
$\land blob_storage' = [blob_storage$ EXCEPT $![b] = [FreeBlob$ EXCEPT
$!.allocated =$ TRUE,
$!.staged =$ TRUE,
$!.req = curr_req[self],$
$!.epoch = curr_epoch[self],$
$!.tenant = jobs[curr_req[self]].tenant,$
$!.query = jobs[curr_req[self]].query,$
$!.version = jobs[curr_req[self]].version,$
$!.security_context = jobs[curr_req[self]].security_context]$
$\land pc' = [pc$ EXCEPT $![self] = \text{“PublishPointer”}]$
ELSE $\land pc' = [pc$ EXCEPT $![self] = \text{“ClaimTask”}]$
$\land$ UNCHANGED $\langle blob_storage,\ available_blobs,$
$curr_blob\rangle$
$\land$ UNCHANGED $\langle catalog_snapshot,\ cache,\ task_queue,$
$jobs,\ worker_alive,\ client_received,$
$commit_count,\ crashes,\ timeouts,$
$req_version,\ curr_req,\ curr_epoch\rangle$

$PublishPointer(self) \triangleq \ \wedge pc[self] = \text{“PublishPointer”}$
$\wedge$ IF $worker_alive[self]$
$\wedge\ HoldsLease(curr_req[self],\ curr_epoch[self],\ self,\ \text{“RUNNING”})$
THEN $\wedge\ jobs' = [jobs$ EXCEPT $![curr_req[self]] = [jobs[curr_req[self]]$ EXCEPT
$!.blob_ptr = curr_blob[self],\ !.state = \text{“RESULT_WRITTEN”}]]$
$\wedge\ blob_storage' = [blob_storage$ EXCEPT $![curr_blob[self]] = [blob_storage[curr_blob[self]]$ EXCEPT $!.staged =$
$\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“CommitJob”}]$
ELSE $\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“ClaimTask”}]$
$\wedge$ UNCHANGED $\langle jobs,\ blob_storage\rangle$
$\wedge$ UNCHANGED $\langle catalog_snapshot,\ cache,\ task_queue,$
$available_blobs,\ worker_alive,$
$client_received,\ commit_count,\ crashes,$
$timeouts,\ req_version,\ curr_req,$
$curr_epoch,\ curr_blob\rangle$

$CommitJob(self) \triangleq \ \wedge pc[self] = \text{“CommitJob”}$
$\wedge$ IF $worker_alive[self]$
$\wedge\ HoldsLease(curr_req[self],\ curr_epoch[self],\ self,\ \text{“RESULT_WRITTEN”})$
THEN $\wedge\ jobs' = [jobs$ EXCEPT $![curr_req[self]] = [jobs[curr_req[self]]$ EXCEPT
$!.state = \text{“COMPLETED”},\ !.owner = NONE]]$
$\wedge\ commit_count' = [commit_count$ EXCEPT $![curr_req[self]] = commit_count[curr_req[self]] + 1]$
$\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“PopulateCache”}]$
ELSE $\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“ClaimTask”}]$
$\wedge$ UNCHANGED $\langle jobs,\ commit_count\rangle$
$\wedge$ UNCHANGED $\langle catalog_snapshot,\ cache,\ task_queue,$
$blob_storage,\ available_blobs,\ worker_alive,$
$client_received,\ crashes,\ timeouts,$
$req_version,\ curr_req,\ curr_epoch,$
$curr_blob\rangle$

$PopulateCache(self) \triangleq \ \wedge pc[self] = \text{“PopulateCache”}$
$\wedge$ IF $worker_alive[self]$
$\wedge\ jobs[curr_req[self]].state = \text{“COMPLETED”}$
$\wedge\ jobs[curr_req[self]].blob_ptr = curr_blob[self]$
$\wedge\ blob_storage[curr_blob[self]].allocated$
$\wedge\ \neg blob_storage[curr_blob[self]].staged$
$\wedge\ BlobMatches(curr_blob[self],\ jobs[curr_req[self]].tenant,\ jobs[curr_req[self]].query,$
$jobs[curr_req[self]].version,\ jobs[curr_req[self]].security_context)$
$\wedge\ jobs[curr_req[self]].version = catalog_snapshot[jobs[curr_req[self]].tenant]$
THEN $\wedge\ cache' = [cache$ EXCEPT $![jobs[curr_req[self]].tenant][jobs[curr_req[self]].query][jobs[curr_req[self]].$
ELSE $\wedge$ TRUE
$\wedge\ cache' = cache$
$\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“ClaimTask”}]$
$\wedge$ UNCHANGED $\langle catalog_snapshot,\ task_queue,\ jobs,$

$blob_storage$, $available_blobs$,
$worker_alive$, $client_received$,
$commit_count$, $crashes$, $timeouts$,
$req_version$, $curr_req$, $curr_epoch$,
$curr_blob\rangle$

$ComputeWorker(self) \triangleq ClaimTask(self) \lor StartExecution(self)$
$\lor AllocateBlob(self) \lor PublishPointer(self)$
$\lor CommitJob(self) \lor PopulateCache(self)$

$LeaseLoop \triangleq \land pc[\text{“LeaseManager”}] = \text{“LeaseLoop”}$
$\land \exists\, r \in ExpirableJobs :$
$\land$ IF $worker_alive[jobs[r].owner]$
THEN $\land\, timeouts' = timeouts + 1$
ELSE $\land$ TRUE
$\land$ UNCHANGED $timeouts$
$\land$ IF $jobs[r].attempts < MaxAttempts$
THEN $\land\, jobs' = [jobs$ EXCEPT $![r] = [jobs[r]$ EXCEPT
$!.owner = NONE,\ !.state = \text{“QUEUED”},$
$!.attempts = @ + 1,\ !.lease_epoch = @ + 1,$
$!.blob_ptr = NONE,\ !.rls_applied =$ FALSE,
$!.executed_predicate_tenant = NONE]]$
$\land\, task_queue' = Append(task_queue, r)$
ELSE $\land\, jobs' = [jobs$ EXCEPT $![r] = [jobs[r]$ EXCEPT
$!.owner = NONE,\ !.state = \text{“FAILED”},$
$!.lease_epoch = @ + 1,\ !.blob_ptr = NONE,$
$!.rls_applied =$ FALSE,
$!.executed_predicate_tenant = NONE]]$
$\land$ UNCHANGED $task_queue$
$\land\, pc' = [pc$ EXCEPT $![\text{“LeaseManager”}] = \text{“LeaseLoop”}]$
$\land$ UNCHANGED $\langle catalog_snapshot, cache, blob_storage,$
$available_blobs, worker_alive, client_received,$
$commit_count, crashes, req_version, curr_req,$
$curr_epoch, curr_blob\rangle$

$LeaseManager \triangleq LeaseLoop$

$GCLoop \triangleq \land pc[\text{“GarbageCollector”}] = \text{“GCLoop”}$
$\land \exists\, b \in GCable :$
$\land\, blob_storage' = [blob_storage$ EXCEPT $![b] = FreeBlob]$
$\land\, available_blobs' = (available_blobs \cup \{b\})$
$\land\, pc' = [pc$ EXCEPT $![\text{“GarbageCollector”}] = \text{“GCLoop”}]$
$\land$ UNCHANGED $\langle catalog_snapshot, cache, task_queue, jobs,$
$worker_alive, client_received, commit_count, crashes,$
$timeouts, req_version, curr_req, curr_epoch,$

$curr_blob\rangle$

$GarbageCollector \triangleq GCLoop$

$EvictLoop \triangleq \land pc[\text{“CacheEvictor”}] = \text{“EvictLoop”}$
$\land \exists\, e \in CachedKeys :$
$cache' = [cache \text{ EXCEPT } ![e[1]][e[2]][e[3]][e[4]] = NoEntry]$
$\land pc' = [pc \text{ EXCEPT } ![\text{“CacheEvictor”}] = \text{“EvictLoop”}]$
$\land \text{UNCHANGED } \langle catalog_snapshot, task_queue, jobs, blob_storage,$
$available_blobs, worker_alive, client_received,$
$commit_count, crashes, timeouts, req_version,$
$curr_req, curr_epoch, curr_blob\rangle$

$CacheEvictor \triangleq EvictLoop$

$CrashLoop \triangleq \land pc[\text{“Crasher”}] = \text{“CrashLoop”}$
$\land \text{IF } crashes < MaxCrashes$
$\text{THEN } \land \exists\, w \in \{x \in Workers : worker_alive[x]\} :$
$\land worker_alive' = [worker_alive \text{ EXCEPT } ![w] = \text{FALSE}]$
$\land crashes' = crashes + 1$
$\land pc' = [pc \text{ EXCEPT } ![\text{“Crasher”}] = \text{“CrashLoop”}]$
$\text{ELSE } \land pc' = [pc \text{ EXCEPT } ![\text{“Crasher”}] = \text{“Done”}]$
$\land \text{UNCHANGED } \langle worker_alive, crashes\rangle$
$\land \text{UNCHANGED } \langle catalog_snapshot, cache, task_queue, jobs,$
$blob_storage, available_blobs, client_received,$
$commit_count, timeouts, req_version, curr_req,$
$curr_epoch, curr_blob\rangle$

$Crasher \triangleq CrashLoop$

$RecoverLoop \triangleq \land pc[\text{“Recoverer”}] = \text{“RecoverLoop”}$
$\land \exists\, w \in \{x \in Workers :$
$\land \neg worker_alive[x]$
$\land \forall\, r \in Requests :$
$\neg(jobs[r].owner = x \land jobs[r].state \in Active)\} :$
$worker_alive' = [worker_alive \text{ EXCEPT } ![w] = \text{TRUE}]$
$\land pc' = [pc \text{ EXCEPT } ![\text{“Recoverer”}] = \text{“RecoverLoop”}]$
$\land \text{UNCHANGED } \langle catalog_snapshot, cache, task_queue, jobs,$
$blob_storage, available_blobs, client_received,$
$commit_count, crashes, timeouts, req_version,$
$curr_req, curr_epoch, curr_blob\rangle$

$Recoverer \triangleq RecoverLoop$

$ETLCommit \triangleq \land pc[\text{“LakehouseETL”}] = \text{“ETLCommit”}$

$\land \exists\, t \in \{x \in \mathit{Tenants} : \mathit{catalog_snapshot}[x] < \mathit{MaxSnapshot}\} :$
$\land\ \mathit{cache}' = [\mathit{cache}$ EXCEPT $![t] = [q \in \mathit{Queries} \mapsto [v \in 1\,..\,\mathit{MaxSnapshot} \mapsto [s \in \mathit{SecCtxs} \mapsto$
IF $v < \mathit{catalog_snapshot}[t] + 1$
THEN $\mathit{NoEntry}$
ELSE $\mathit{cache}[t][q][v][s]]]]$
$\land\ \mathit{catalog_snapshot}' = [\mathit{catalog_snapshot}$ EXCEPT $![t] = \mathit{catalog_snapshot}[t] + 1]$
$\land\ \mathit{pc}' = [\mathit{pc}$ EXCEPT $!["\mathsf{LakehouseETL}"] = "\mathsf{ETLCommit}"]$
$\land$ UNCHANGED $\langle \mathit{task_queue},\ \mathit{jobs},\ \mathit{blob_storage},\ \mathit{available_blobs},$
$\mathit{worker_alive},\ \mathit{client_received},\ \mathit{commit_count},$
$\mathit{crashes},\ \mathit{timeouts},\ \mathit{req_version},\ \mathit{curr_req},$
$\mathit{curr_epoch},\ \mathit{curr_blob}\rangle$

$\mathit{LakehouseETL} \triangleq \mathit{ETLCommit}$

$\mathit{Next} \triangleq \mathit{LeaseManager} \lor \mathit{GarbageCollector} \lor \mathit{CacheEvictor} \lor \mathit{Crasher}$
$\lor\ \mathit{Recoverer} \lor \mathit{LakehouseETL}$
$\lor\ (\exists\, \mathit{self} \in \mathit{Requests} : \mathit{ClientProcess}(\mathit{self}))$
$\lor\ (\exists\, \mathit{self} \in \mathit{Workers} : \mathit{ComputeWorker}(\mathit{self}))$

$\mathit{Spec} \triangleq\ \land\, \mathit{Init} \land \Box[\mathit{Next}]_{\mathit{vars}}$
$\land\ \forall\, \mathit{self} \in \mathit{Requests} : \mathrm{WF}_{\mathit{vars}}(\mathit{ClientProcess}(\mathit{self}))$
$\land\ \forall\, \mathit{self} \in \mathit{Workers} : \mathrm{WF}_{\mathit{vars}}(\mathit{ComputeWorker}(\mathit{self}))$
$\land\ \mathrm{WF}_{\mathit{vars}}(\mathit{LeaseManager})$
$\land\ \mathrm{WF}_{\mathit{vars}}(\mathit{GarbageCollector})$
$\land\ \mathrm{WF}_{\mathit{vars}}(\mathit{CacheEvictor})$
$\land\ \mathrm{WF}_{\mathit{vars}}(\mathit{Recoverer})$
$\land\ \mathrm{WF}_{\mathit{vars}}(\mathit{LakehouseETL})$

$\mathit{EpochMonotonic} \triangleq$
$\Box[\forall\, r \in \mathit{Requests} : \mathit{jobs}'[r].\mathit{lease_epoch} \geq \mathit{jobs}[r].\mathit{lease_epoch}]_{\mathit{vars}}$

$\mathit{SnapshotNeverChanges} \triangleq$
$\Box[\forall\, r \in \mathit{Requests} :$
$(\mathit{jobs}[r].\mathit{state} \neq "\mathsf{IDLE}" \land \mathit{jobs}'[r].\mathit{state} \neq "\mathsf{IDLE}") \Rightarrow$
$\mathit{jobs}'[r].\mathit{version} = \mathit{jobs}[r].\mathit{version}]_{\mathit{vars}}$

$\mathit{CatalogMonotonic} \triangleq$
$\Box[\forall\, t \in \mathit{Tenants} : \mathit{catalog_snapshot}'[t] \geq \mathit{catalog_snapshot}[t]]_{\mathit{vars}}$

$\mathit{TerminalAbsorbing} \triangleq$

$$\Box[\forall\, r \in \mathit{Requests} :$$
$$\mathit{jobs}[r].\mathit{state} \in \mathit{Terminal} \Rightarrow \mathit{jobs}'[r].\mathit{state} = \mathit{jobs}[r].\mathit{state}]_{\mathit{vars}}$$

$$\mathit{CommitMonotonic} \triangleq$$
$$\Box[\forall\, r \in \mathit{Requests} : \mathit{commit_count}'[r] \geq \mathit{commit_count}[r]]_{\mathit{vars}}$$

$$\mathit{DeliveryImmutable} \triangleq$$
$$\Box[\forall\, r \in \mathit{Requests} :$$
$$\mathit{client_received}[r].\mathit{delivered} \Rightarrow$$
$$\land\ \mathit{client_received}'[r].\mathit{delivered}$$
$$\land\ \mathit{client_received}'[r].\mathit{blob_id} = \mathit{client_received}[r].\mathit{blob_id}$$
$$\land\ \mathit{client_received}'[r].\mathit{version} = \mathit{client_received}[r].\mathit{version}]_{\mathit{vars}}$$

$$\mathit{LiveTermination} \triangleq$$
$$\forall\, r \in \mathit{Requests} :$$
$$(\mathit{jobs}[r].\mathit{state} \in \mathit{Pending}) \rightsquigarrow (\mathit{jobs}[r].\mathit{state} \in \mathit{Terminal})$$

$$\mathit{ClientsComplete} \triangleq$$
$$\forall\, r \in \mathit{Requests} : \Diamond(\mathit{client_received}[r].\mathit{released})$$

$$\mathit{GCReclaimsGarbage} \triangleq$$
$$\forall\, b \in \mathit{Blobs} : (b \in \mathit{GCable}) \rightsquigarrow (b \notin \mathit{GCable})$$